Actividad 3: Arquitectura, modelo de datos, UI/UX y prototipo navegable

Proyecto: Gestor de Tareas Académicas. En esta fase todavía no se programa la app; lo que hice fue dejar bien definido cómo va a estar construida (arquitectura y tecnologías), qué datos va a guardar (modelo y diagrama ER) y cómo se va a ver y navegar (wireframes, guía de estilo y un prototipo que se puede recorrer).

- **Repositorio:** https://github.com/esteban-lt/academic_task_manager
- **Prototipo navegable:** https://esteban-lt.github.io/academic_task_manager/design/prototype/index.html

El diseño y el prototipo se elaboraron en HTML/CSS con apoyo de Claude.

1. Documento de arquitectura

1.1 Descripción general del proyecto

- **Problema:** a muchos estudiantes se les juntan tareas, proyectos y exámenes de varias materias, y los apuntan en lugares distintos (la libreta, el chat del grupo, la plataforma de cada profe). Así es fácil que se pase una fecha o que no se note qué es lo más urgente.
- **Objetivo:** una app sencilla donde el alumno registre cada pendiente con su materia, tipo, urgencia y fecha de entrega, lo vea ordenado en una lista y lo borre cuando ya no le sirva. También puede marcarlo como importante, activar un recordatorio y estimar el esfuerzo de 1 a 5.
- **Público objetivo:** estudiantes de preparatoria y universidad que usan el celular todo el día y llevan varias materias a la vez. Como uso secundario, cualquier persona que quiera organizar entregas por curso.

Funcionalidad meta de este bloque (del contexto del mini-proyecto): registrar tarea (título, materia, tipo, urgencia y fecha), listar tareas y eliminar tareas.

1.2 Selección del tipo de arquitectura

Elegí una arquitectura híbrida (multiplataforma) con Flutter: un solo código en Dart que se compila a app nativa de Android (y después de iOS o web). Comparé las tres opciones contra lo que pide el proyecto:

Criterio	Nativa	Web	Híbrida (Flutter)
Requerimientos (formularios, listas, fecha, recordatorio)	Sí, pero dos códigos	Sí, notificaciones limitadas	Sí, un solo código
Dispositivos (Android ahora, iOS después)	Un proyecto por sistema	Cualquier navegador	Android, iOS y web
Recursos de desarrollo (1 persona, Windows)	Kotlin + Swift (Swift pide Mac)	HTML/CSS/JS	Solo Dart; ya tengo Flutter instalado
Tiempo (un semestre)	El doble	Rápido	Rápido, con hot reload
Despliegue	Dos tiendas	Un servidor	APK / Play Store; web opcional

La nativa da el mejor rendimiento, pero para una app de formularios y listas no hace falta y duplicaría el trabajo. La web se despliega fácil, pero se siente menos como app y los recordatorios en el celular quedan

limitados. Flutter queda en medio: un solo código, widgets de Material Design listos (los mismos del curso: `Form`, `TextFormField`, `DropdownButtonFormField`, `Slider`, `showDatePicker`) y además es lo que ya uso en la materia, así que no hay curva de aprendizaje extra.

Los datos van en la nube para que no se pierdan si cambias de teléfono. La app no se conecta directo a la base de datos: si lo hiciera, tendría que llevar el usuario y la contraseña de Postgres dentro del APK, y cualquiera podría sacarlos. Por eso en medio hay una API REST que es la única que conoce esas credenciales.

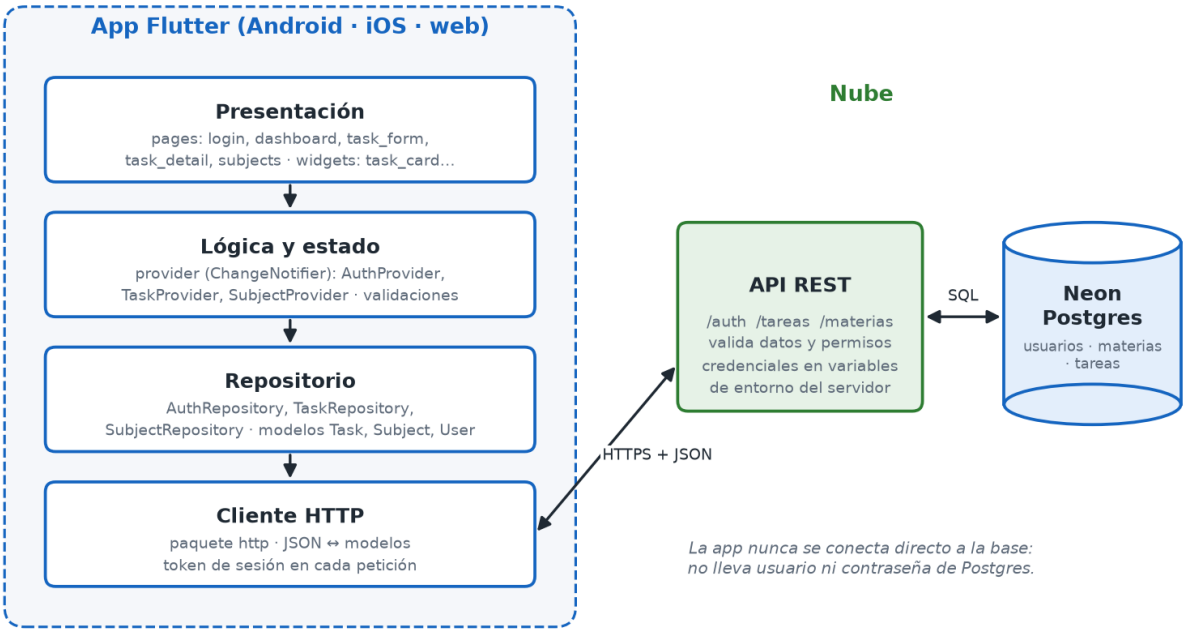


Figura 1. Arquitectura en capas

Capa	Qué hace
Presentación	Pantallas (`pages/`) y widgets reutilizables (`widgets/`). Solo dibujan y avisan de lo que tocó el usuario.
Lógica y estado	`provider` con `ChangeNotifier`: guarda la lista de tareas y materias, valida y avisa a las pantallas cuando algo cambia.
Repositorio	Una clase por entidad (`TaskRepository`, `SubjectRepository`, `AuthRepository`). Convierte entre JSON y modelos; si mañana cambia la API, solo se toca aquí.
Cliente HTTP	Paquete `http`: hace las peticiones a la API con el token de sesión.
API REST (nube)	Valida datos y permisos (cada quien ve solo sus tareas) y habla con la base. Sus credenciales viven en variables de entorno del servidor.
Neon Postgres (nube)	Base de datos relacional con las tablas `usuarios`, `materias` y `tareas`.

Ejemplo del flujo al guardar una tarea: el formulario valida los campos → `TaskProvider.add()` → `TaskRepository.create()` arma el JSON → `POST /tareas` → la API valida y hace el `INSERT` en Postgres → responde la tarea creada → el provider la agrega a la lista y la pantalla se redibuja con el aviso "Tarea guardada".

Método y ruta	Para qué
POST /auth/login	Iniciar sesión; regresa el token
GET /tareas	Listar las tareas del usuario (orden por fecha de entrega)
POST /tareas	Registrar una tarea
GET /tareas/{id}	Ver el detalle
DELETE /tareas/{id}	Eliminar una tarea
GET /materias · POST /materias	Listar y agregar materias

1.3 Entorno y tecnologías

Aspecto	Elección
Lenguaje y framework	Dart 3 con Flutter 3 (Material 3)
Entorno de desarrollo	Windows 11, Android Studio con el SDK de Android y su emulador; Flutter SDK por terminal
Control de versiones	Git con commits pequeños por tema y repositorio público en GitHub (https://github.com/esteban-lt/academic_task_manager)
Dependencias principales	`provider` (estado), `http` (llamadas a la API), `intl` (formato de fechas en español). Pocas a propósito.
Backend (propuesta)	API REST, por ejemplo Node.js + Express, desplegada en un servicio en la nube
Base de datos	Neon Postgres (Postgres administrado en la nube). En esta fase solo está diseñada; no se creó cuenta ni conexión.
Diseño	Wireframes, guía de estilo y prototipo hechos en HTML/CSS (sin Figma), publicados con GitHub Pages

1.4 Modelo de datos y diagrama ER

Son tres entidades. Usé los tipos de Postgres: `UUID` para las llaves (se pueden generar sin chocar), `VARCHAR` con largo máximo, `TIMESTAMPTZ` para fechas con zona horaria, `BOOLEAN` para los sí/no y `SMALLINT` con `CHECK` para el esfuerzo.

Entidad	Qué representa
Usuario (`usuarios`)	El alumno que usa la app. Inicia sesión con su correo.
Materia (`materias`)	Una materia del alumno (nombre, profesor y un color para reconocerla en la lista).
Tarea (`tareas`)	Cada pendiente: título, tipo, urgencia, fecha de entrega, importante, recordatorio y esfuerzo.

Campos principales

Tabla	Campo	Tipo	Restricción / nota
usuarios	id	UUID	PK
usuarios	nombre	VARCHAR(100)	NOT NULL
usuarios	correo	VARCHAR(150)	NOT NULL, UNIQUE
usuarios	password_hash	VARCHAR(255)	NOT NULL (nunca la contraseña en texto plano)
materias	id · usuario_id	UUID	PK · FK → usuarios
materias	nombre · profesor	VARCHAR(100)	nombre NOT NULL; UNIQUE (usuario_id, nombre)
materias	color	VARCHAR(7)	Hex, por defecto #1565C0
tareas	id · usuario_id · materia_id	UUID	PK · FK → usuarios · FK → materias
tareas	titulo	VARCHAR(150)	NOT NULL, mínimo 3 caracteres (CHECK)
tareas	tipo	VARCHAR(10)	CHECK: Tarea, Proyecto o Examen
tareas	urgencia	VARCHAR(5)	CHECK: Baja, Media o Alta
tareas	fecha_entrega	TIMESTAMPTZ	Puede quedar vacía
tareas	importante · recordatorio	BOOLEAN	DEFAULT false
tareas	esfuerzo	SMALLINT	CHECK entre 1 y 5, DEFAULT 3
(todas)	creado_en	TIMESTAMPTZ	DEFAULT now()

Relaciones

- **Usuario 1:N Tarea:** un usuario registra muchas tareas; cada tarea es de un solo usuario. Si se borra el usuario, se borran sus tareas (ON DELETE CASCADE).
- **Materia 1:N Tarea:** una materia agrupa muchas tareas; cada tarea pertenece a una materia. No se puede borrar una materia que todavía tiene tareas (ON DELETE RESTRICT), para no perder pendientes por accidente.
- **Usuario 1:N Materia (agregada):** la agregué porque si no, todos los usuarios compartirían la misma lista de materias. Así cada alumno tiene las suyas, y no puede repetir el nombre de una materia.

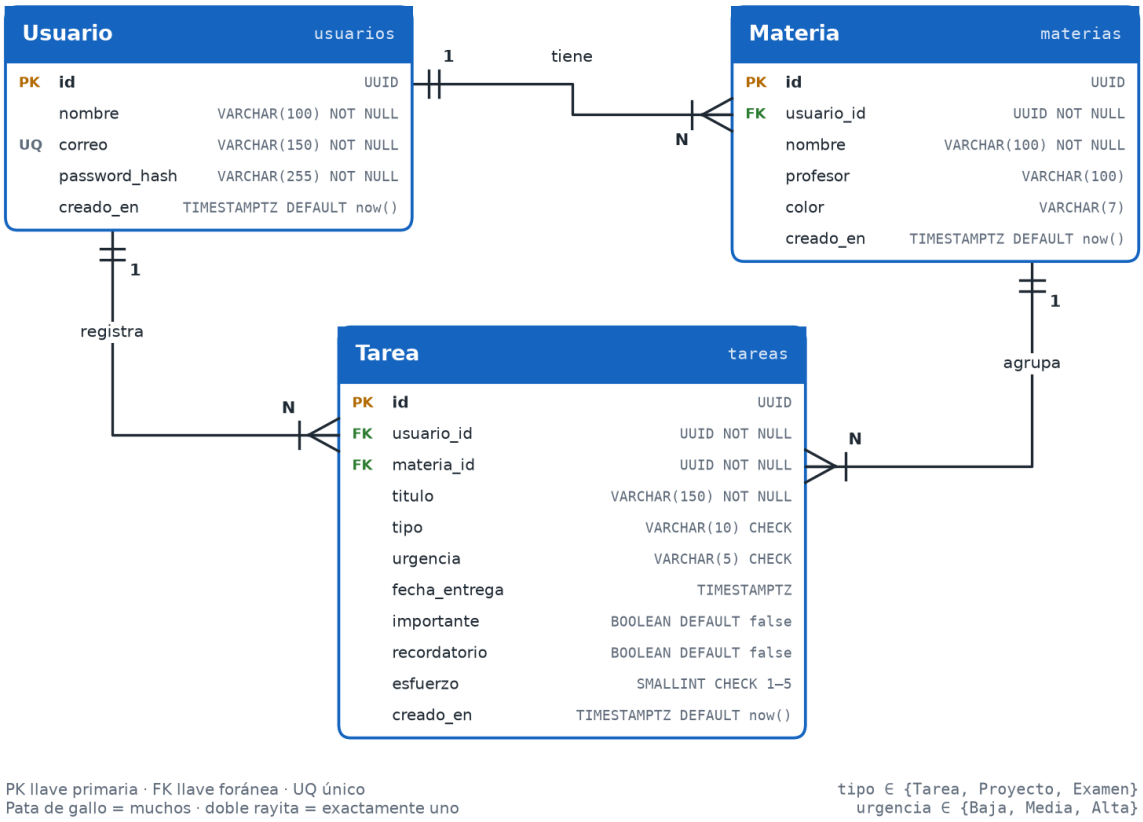


Figura 2. Diagrama ER (notación pata de gallo)

El modelo sale directo del `Task` de la práctica guiada (3.2 y 3.3); solo cambié los nombres a español y volví la materia una entidad propia en lugar de un texto suelto:

Campo en Task (práctica)	Columna en tareas
title	titulo
subject	materia_id → materias.nombre
type	tipo
urgency (solo UI en la práctica)	urgencia (ahora sí se guarda)
dueDate	fecha_entrega
important · reminder	importante · recordatorio
effort	esfuerzo
createdAt	creado_en

docs/modelo_datos.sql (tabla tareas; el archivo completo está en el repo)

```
-- Usuario 1:N Tarea y Materia 1:N Tarea.
CREATE TABLE tareas (
  id             UUID         PRIMARY KEY DEFAULT gen_random_uuid(),
  usuario_id     UUID         NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,
  materia_id     UUID         NOT NULL REFERENCES materias(id) ON DELETE RESTRICT,
  titulo         VARCHAR(150) NOT NULL CHECK (char_length(trim(titulo)) >= 3),
  tipo           VARCHAR(10)  NOT NULL DEFAULT 'Tarea'
                 CHECK (tipo IN ('Tarea', 'Proyecto', 'Examen')),
  urgencia       VARCHAR(5)   NOT NULL DEFAULT 'Media'
                 CHECK (urgencia IN ('Baja', 'Media', 'Alta')),
  fecha_entrega  TIMESTAMPTZ,
  importante     BOOLEAN      NOT NULL DEFAULT false,
  recordatorio   BOOLEAN      NOT NULL DEFAULT false,
  esfuerzo       SMALLINT     NOT NULL DEFAULT 3 CHECK (esfuerzo BETWEEN 1 AND 5),
  creado_en      TIMESTAMPTZ  NOT NULL DEFAULT now()
);
```

2. UI/UX: diseño visual y navegación

2.1 Wireframes

Son cinco pantallas más el diálogo para confirmar al eliminar. Los wireframes son la misma base HTML del prototipo, pero en escala de grises, con Arial, sin sombras y con bordes punteados: así se ve solo la estructura (qué va dónde y qué pesa más) sin distraerse con colores. Este es el mapa de navegación:

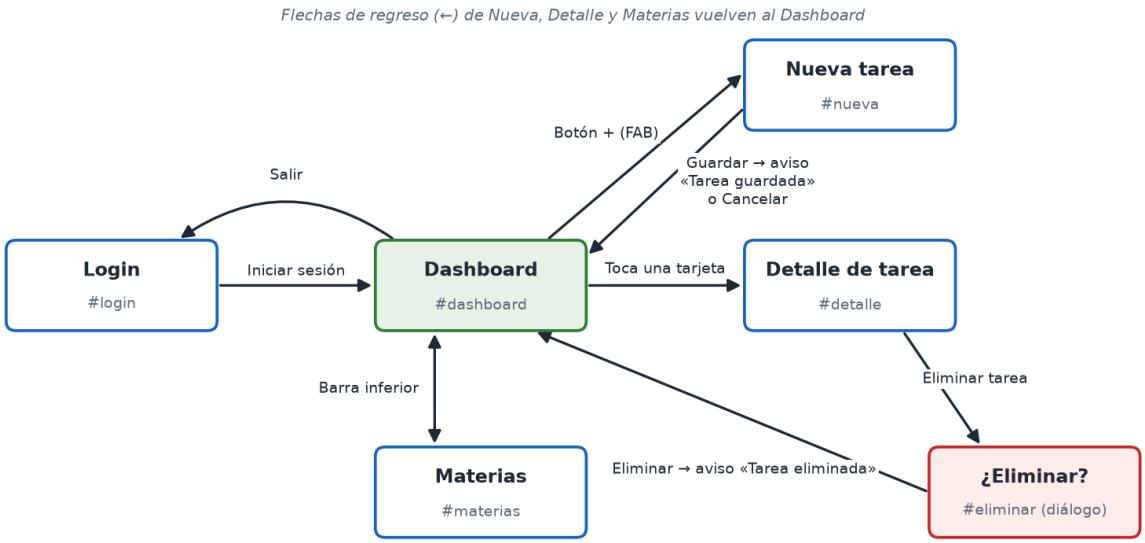
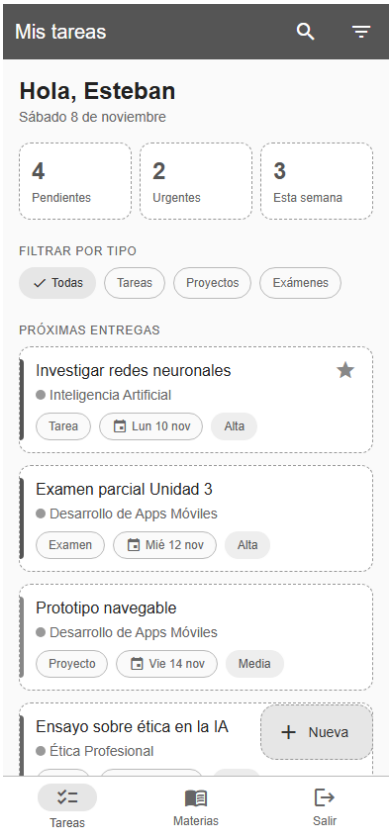


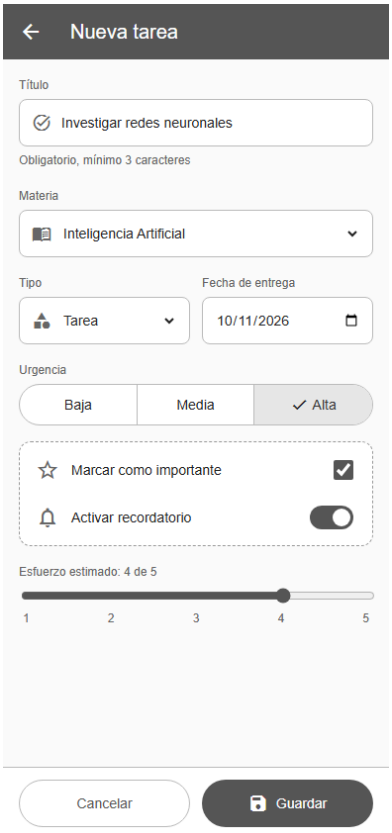
Figura 3. Navegación entre pantallas



Login: logo, título, correo, contraseña y un solo botón principal



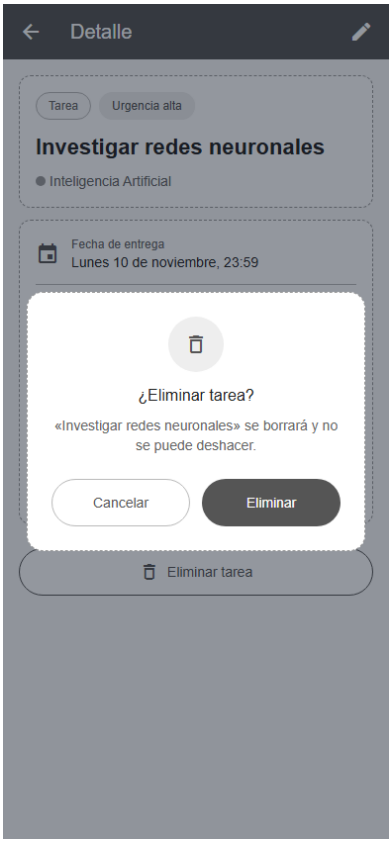
Dashboard: resumen, filtros por tipo, lista de tarjetas, FAB y barra inferior



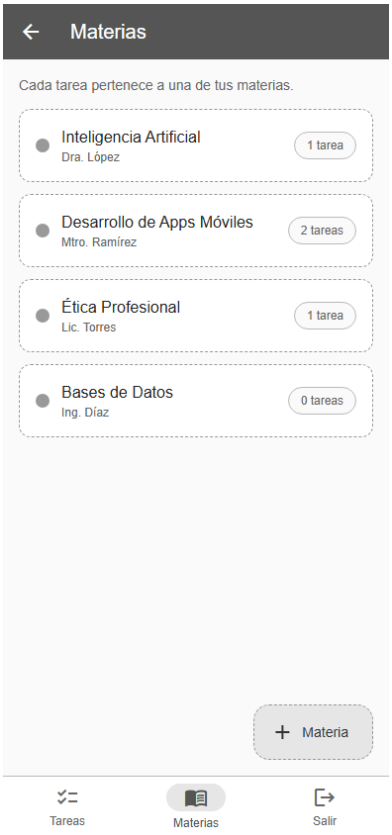
Nueva tarea: los 8 campos en una columna; Guardar fijo abajo



Detalle: datos de la tarea y botón de eliminar al final



Confirmación antes de eliminar



Materias: lista con número de tareas y FAB para agregar

Algunas decisiones de jerarquía: lo más importante de cada tarjeta es el título (más grande), luego la materia y al final los chips; la urgencia se ve de reojo por la barra de color a la izquierda. La acción principal de cada pantalla es un solo botón relleno (Iniciar sesión, Guardar, el FAB), y la acción peligrosa (eliminar) va al final, con contorno rojo y siempre pide confirmación.

2.2 Guía de estilo

La guía completa está en `docs/guia_estilo.md` del repo. Es exactamente la que usa el prototipo: los valores salen de las variables `:root` de `styles.css`. Este es el resumen:

	Color	Hex	Uso
	Primario	#1565C0	App bar, botón principal, enlaces
	Primario oscuro	#0D47A1	Texto sobre contenedor primario
	Contenedor primario	#E3EEFB	FAB, chip y pestaña seleccionados
	Secundario (verde)	#2E7D32	Urgencia baja, estados positivos
	Acento (ámbar)	#F9A825	Urgencia media, estrella de importante
	Peligro (rojo)	#C62828	Urgencia alta, eliminar
	Fondo	#F5F7FA	Fondo de pantallas
	Texto	#1F2933	Texto principal
	Texto secundario	#5F6B7A	Etiquetas, fechas y ayudas

Medí el contraste de cada combinación de texto: todas pasan el nivel AA (4.5 : 1). Por eso los chips de urgencia baja y media usan un verde y un ámbar más oscuros para el texto (`#1B5E20` y `#8A5300`) que para las barras.

Elemento	Regla
Tipografía	Roboto. Encabezado 24/32 px negrita · título de barra 20/28 px · subtítulo 16/24 px · cuerpo 14/20 px · leyenda 12/16 px
Iconografía	Material Icons rellenos, 24 px (20 px en campos y botones, 16 px en chips). Ej.: checklist, menu_book, add, event, star, delete_outline
Botones	Principal relleno azul en píldora de 48 px; secundario con contorno; peligro en rojo; FAB extendido de 56 px con radio 16
Campos	48 px de alto, borde gris de 1 px y radio 8; al enfocar, borde azul de 2 px; ícono a la izquierda y ayuda abajo
Espaciado	Escala de 4 px: 4, 8, 12, 16, 24, 32. Margen de pantalla 16 px; entre tarjetas 12; entre secciones 24
Bordes y sombras	Radio 8 campos · 12 tarjetas · 16 FAB y diálogos · píldora en botones y chips. Sombra nivel 1 en tarjetas, nivel 2 en FAB y diálogos
Layout	Pantalla de 390 × 844 px: app bar 56 px, contenido con scroll, barra inferior 64 px. Áreas táctiles de 48 px mínimo

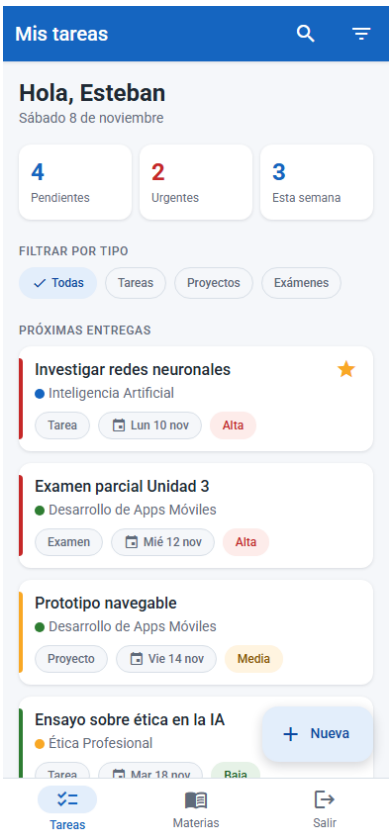
3. Prototipo navegable

Está publicado en https://esteban-lt.github.io/academic_task_manager/design/prototype/index.html y se puede abrir desde el celular o la computadora. Es un solo archivo HTML donde cada pantalla es una sección; los botones son enlaces a `#login`, `#dashboard`, `#nueva`, `#detalle` y `#materias`, y el selector `:target` de CSS muestra solo la sección elegida. No hay base de datos todavía, así que guardar y eliminar se simulan con un aviso, como permite la actividad. Agregando `?wireframe` a la URL se ve la versión de wireframes con la misma navegación.

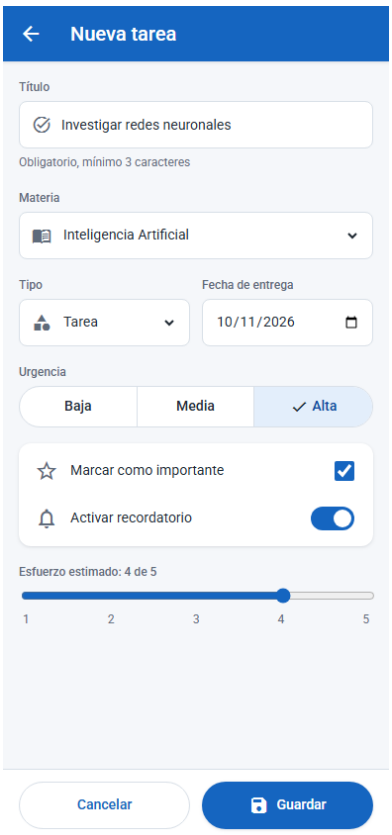
Desde	Acción	Llega a
Login	Iniciar sesión	Dashboard
Dashboard	Botón + Nueva	Nueva tarea
Nueva tarea	Guardar (aviso «Tarea guardada») o Cancelar	Dashboard
Dashboard	Tocar una tarjeta	Detalle
Detalle	Eliminar tarea	Diálogo de confirmación
Diálogo	Eliminar (aviso «Tarea eliminada») / Cancelar	Dashboard / Detalle
Dashboard ↔ Materias	Barra inferior	Materias / Tareas
Cualquiera	Salir (barra inferior) o flecha ←	Login / Dashboard



Login



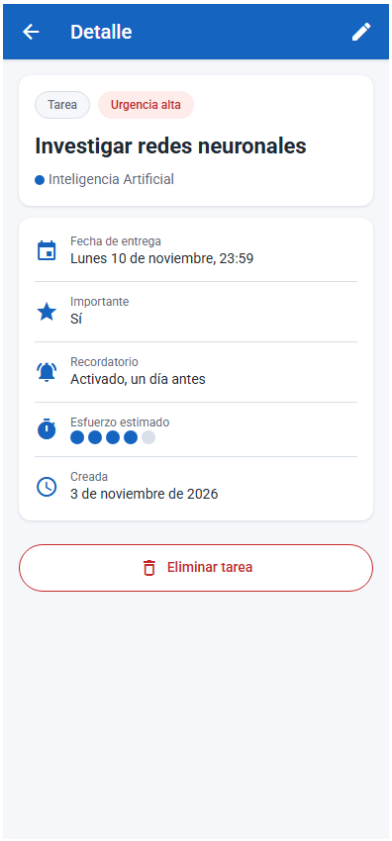
Dashboard



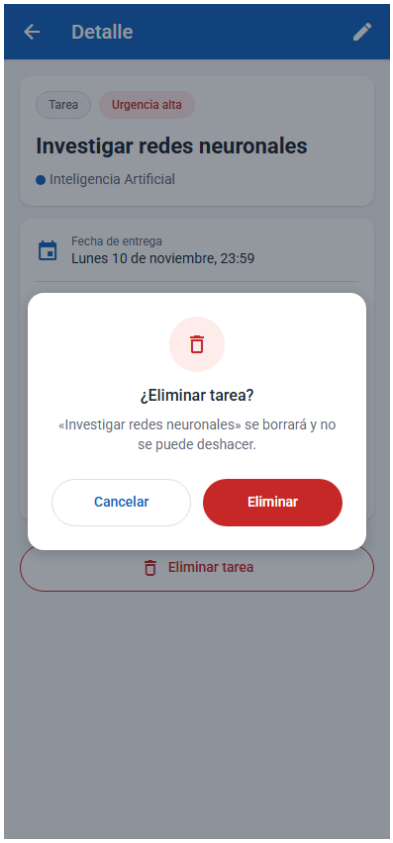
Nueva tarea con todos los controles



Al guardar: aviso «Tarea guardada»



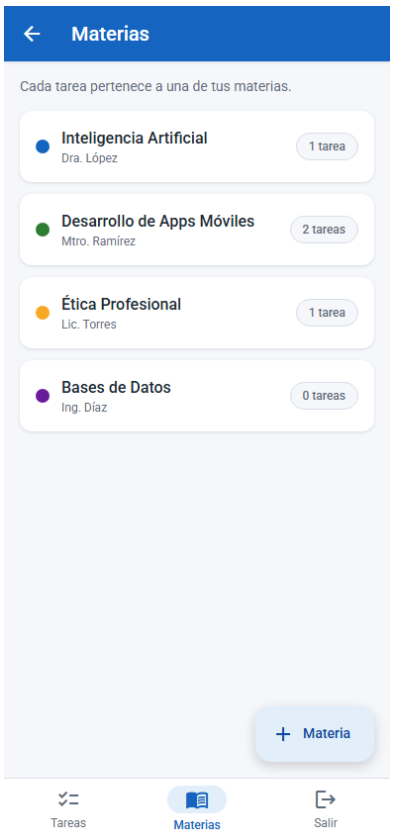
Detalle de la tarea



Confirmación al eliminar



Al eliminar: aviso «Tarea eliminada»



Materias

4. Evidencias en el repositorio

```
academic_task_manager/
├─ README.md                 enlaces al prototipo y a este PDF
├─ docs/
│  ├─ Actividad_3_Unidad_3.pdf   este documento
│  ├─ guia_estilo.md             guía de estilo
│  ├─ diagrama_er.png            diagrama ER
│  ├─ arquitectura_capas.png · navegacion.png
│  ├─ modelo_datos.sql           tablas en Postgres (solo diseño)
│  ├─ fuentes/                   scripts de Python de los diagramas
│  └─ material/                  enunciado y contexto del mini-proyecto
└─ design/
   ├─ prototype/                 index.html + styles.css (fuente del diseño)
   ├─ wireframes/                capturas en escala de grises
   ├─ prototipo/                 capturas del prototipo
   └─ capturas.sh                saca las capturas con Chrome sin ventana
```

Como no usé Figma, los "archivos fuente" del diseño son el HTML y el CSS de `design/prototype/`. Los commits son pequeños y cada uno dice qué parte de la fase agrega:

Commit	Mensaje
f911066	docs: estructura del repositorio y material de la actividad
5606a91	docs: modelo de datos y diagrama ER
41da586	docs: diagramas de arquitectura en capas y de navegación
868f805	design: prototipo navegable de las 5 pantallas en HTML/CSS
4ac0b11	docs: guía de estilo
12c290a	design: capturas de wireframes y del prototipo

Resumen frente a la rúbrica

Criterio	Dónde está
Documento de arquitectura	Sección 1: descripción, capas, flujo, endpoints y tecnologías
Tipo de arquitectura y justificación	Sección 1.2: híbrida con Flutter, comparada contra nativa y web por requerimientos, dispositivos, recursos, tiempo y despliegue
Modelo de datos + ER	Sección 1.4: entidades, campos con tipos de Postgres, relaciones con cardinalidad y diagrama ER
Wireframes	Sección 2.1: 5 pantallas + diálogo, mapa de navegación
Guía de estilo	Sección 2.2 y docs/guia_estilo.md
Prototipo navegable	Sección 3: https://esteban-lt.github.io/academic_task_manager/design/prototype/index.html
Evidencias en repositorio	Sección 4: carpetas docs/ y design/, commits descriptivos